

# DL Midsem Notes — Section 6

## Feedforward Neural Networks and Backpropagation

### 1 Theory

#### 1.1 Architecture and Notation

A feedforward network stacks layers of neurons. For a neuron receiving inputs  $x_1, \dots, x_n$  with weights  $w_1, \dots, w_n$  and bias  $b$ :

$$a = \sum_{i=1}^n w_i x_i + b, \quad h = \sigma(a)$$

$a$  is the *pre-activation*,  $h$  is the *post-activation*. Stacked over layer  $L$ :  $a_L = W_L h_{L-1} + b_L$ ,  $h_L = \sigma(a_L)$ , with  $h_0 = x$ .

**Notation you will see used interchangeably in slides / papers:**

- Bias may appear as an additive term  $b$ , or as a threshold  $\theta$  with  $a = \sum w_i x_i - \theta$  (older MP/perceptron notation).
- The network's predicted output is written  $\hat{y}$  (slides) or  $y'$  (some exam papers) — same quantity.
- Loss/cost is written  $L(\theta)$ ,  $J$ ,  $E$ , or *err* depending on the source — same quantity.

Do not let a paper's choice of symbol confuse you; always check what is being summed/compared.

#### 1.2 Loss Functions

**Mean Squared Error** (regression, or classification when you want a simple example):

$$L_{\text{MSE}} = \frac{1}{2}(y - \hat{y})^2$$

**Binary Cross-Entropy** (binary classification,  $\hat{y} \in (0, 1)$ ):

$$L_{\text{BCE}} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

**Why not MSE for classification?** With a sigmoid output and MSE, the gradient contains a factor  $\hat{y}(1-\hat{y})$  which vanishes when  $\hat{y}$  is confidently wrong (saturated), so learning stalls exactly when it is needed most. MSE loss surfaces for classification are also non-convex with poor curvature near the decision boundary. BCE's gradient with a sigmoid output cancels this saturation term cleanly (see Q4 for the numbers).

#### 1.3 Forward Pass

For a network  $x \rightarrow h$  (hidden)  $\rightarrow \hat{y}$  (output), with sigmoid activations throughout:

$$a_h = W_1 x + b_1, \quad h = \sigma(a_h), \quad a_y = W_2 h + b_2, \quad \hat{y} = \sigma(a_y)$$

where  $\sigma(z) = \frac{1}{1+e^{-z}}$  and the useful identity  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ .

## 1.4 What Backpropagation Actually Computes

Forget formulas for a second. A feedforward network is a **computational graph**: inputs flow through a chain of simple operations (weighted sum, then activation, then weighted sum, ...) until a single number, the loss, comes out. Backprop is just **reverse-mode differentiation on that graph** — a mechanical way to get  $\partial L / \partial w$  for *every* weight in one backward sweep, instead of re-deriving each gradient from scratch.

Two facts make this possible:

1. **The forward pass already computed everything backprop needs.** Every  $a$  (pre-activation) and  $h$  (post-activation) at every neuron got computed once, going forward. Backprop reuses these cached numbers — it never recomputes the forward pass.
2. **The chain rule turns a long product into a short recursion.**  $\partial L / \partial w$  for a weight buried in layer 2 is a product of local derivatives along the path from that weight to the loss: (local slope at layer 2)  $\times$  (local slope at layer 3)  $\times \dots \times$  (local slope at the loss). Rather than writing that whole product out for every weight separately, backprop computes it *layer by layer, back to front*, carrying forward only one running number per neuron — its **error signal**  $\delta$ .

**Definition.**  $\delta$  at a neuron is  $\frac{\partial L}{\partial a}$  — how much the loss would change if that neuron’s pre-activation nudged up slightly. It bundles up "everything downstream" into one number, so that to get the gradient for a weight *feeding into* that neuron, you only ever need: *this neuron’s  $\delta \times$  whatever value flowed into that weight*. That’s the entire trick. The reason it must run backward: to compute a neuron’s  $\delta$  you need the  $\delta$ ’s of every neuron it feeds *forward* into, which only exist once those have already been computed. So the computation order is forced to be output  $\rightarrow \dots \rightarrow$  input, opposite to the forward pass.

## 1.5 The Backprop Recursion (Any Activation)

Nothing above depended on sigmoid — swap in  $\varphi$  for whatever activation you’re using and only one factor changes: the local slope  $\varphi'(a)$ . This is the point most people miss: **the algorithm is identical regardless of activation function**; only the number you plug in for "local slope" changes. Writing  $\varphi$  for a generic activation:

**Output layer:**

$$\delta_{\text{out}} = \frac{\partial L}{\partial \hat{y}} \cdot \varphi'(a_y)$$

(for MSE,  $\partial L / \partial \hat{y} = \hat{y} - y$ ; this factor changes with the loss, not the activation.)

**Hidden layer** (neuron  $j$  feeding forward through weights  $w_{jk}$  to downstream neurons  $k$ ):

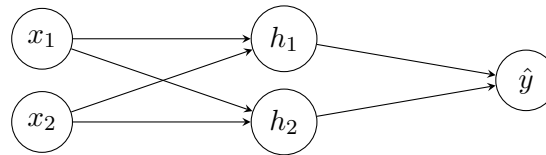
$$\delta_{h_j} = \left( \sum_k \delta_k w_{jk} \right) \varphi'(a_{h_j})$$

Read this as: *gather blame from everything you feed into (weighted by how strongly you feed into it), then scale by your own local sensitivity.*

**Weight and bias gradients (any layer, any activation):**

$$\frac{\partial L}{\partial w_{ij}} = \delta_j \cdot (\text{input to that weight}), \quad \frac{\partial L}{\partial b_j} = \delta_j$$

**SGD update** with learning rate  $\eta$ :  $w \leftarrow w - \eta \partial L / \partial w$ .



hidden layer

## 1.6 $\varphi'$ for Common Activations

You'll see all of these used as the "local slope" factor in the recursion above. (Full trade-offs between them are section 8's topic — this is just what you plug into backprop.)

| Activation | $\varphi(a)$                     | $\varphi'(a)$                                           |
|------------|----------------------------------|---------------------------------------------------------|
| Sigmoid    | $\frac{1}{1 + e^{-a}}$           | $\varphi(a)(1 - \varphi(a))$                            |
| Tanh       | $\tanh(a)$                       | $1 - \tanh^2(a)$                                        |
| ReLU       | $\max(0, a)$                     | 1 if $a > 0$ , else 0 (take 0 at $a = 0$ by convention) |
| Leaky ReLU | $a$ if $a > 0$ , else $\alpha a$ | 1 if $a > 0$ , else $\alpha$                            |

**Why this matters in practice:** sigmoid and tanh squash their derivative toward 0 when  $|a|$  is large (saturation  $\rightarrow$  vanishing gradients through many layers). ReLU's derivative is a clean 1 or 0 — cheap and non-vanishing when active, but a neuron that lands at  $a < 0$  contributes *exactly zero* gradient to everything upstream of it (the "dead ReLU" problem — see Q6).

## 1.7 Parameter Counting

For a network with  $n$  inputs, one hidden layer of  $k$  neurons, and  $m$  outputs, with a bias at every neuron:

$$\# \text{params} = \underbrace{k(n+1)}_{\text{input} \rightarrow \text{hidden}} + \underbrace{m(k+1)}_{\text{hidden} \rightarrow \text{output}}$$

## 2 Practice Questions

**Q1 (Numerical, PYQ-style).** A network has inputs  $x_1 = 0.6$ ,  $x_2 = 0.1$ , two hidden neurons  $h_1, h_2$ , and one output neuron  $\hat{y}$ , all with sigmoid activation. Initial weights:

$$W_1(x_1 \rightarrow h_1) = 0.2, \quad W_2(x_2 \rightarrow h_1) = 0.4, \quad W_3(b_1) = 0.3,$$

$$W_4(x_1 \rightarrow h_2) = 0.1, \quad W_5(x_2 \rightarrow h_2) = 0.5, \quad W_6(b_2) = 0.3,$$

$$W_7(h_1 \rightarrow \hat{y}) = 0.3, \quad W_8(h_2 \rightarrow \hat{y}) = 0.6, \quad b_o = 0.2.$$

Target  $y = 0.3$ , loss is MSE, learning rate  $\eta = 0.1$ . Compute the updated values of  $W_4$ ,  $W_7$ ,  $W_8$  after one epoch of SGD.

**Q2 (Derivation, PYQ-style).** Consider inputs  $x_1, x_2$  feeding a single hidden sigmoid neuron  $h$  via weights  $w_1, w_2$  (ignore hidden bias), which feeds a sigmoid output neuron  $\hat{y}$  via weight  $w_3$  (ignore output bias). Loss is  $L = \frac{1}{2}(y - \hat{y})^2$ . Derive the generic expression for  $\frac{\partial L}{\partial w_i}$ , then write out  $\text{grad}(w_1)$ ,  $\text{grad}(w_2)$ ,  $\text{grad}(w_3)$  explicitly.

**Q3 (Numerical + general formula).** A 1-hidden-layer network has  $n$  inputs,  $k$  hidden neurons,  $m$  outputs, with bias at every neuron. (a) Derive the total parameter count in terms of  $n, k, m$ . (b) Evaluate it for  $n = 4$ ,  $k = 5$ ,  $m = 2$ .

**Q4 (PYQ-style, loss functions).** You are building a binary classifier (sweet watermelon = 1, not sweet = 0).

- (i) Give one reason MSE is a poor loss choice here.
- (ii) Write the BCE loss formula for a single example, in terms of  $y$  and  $\hat{y}$ .
- (iii) Compute the loss for a prediction  $\hat{y} = 0.9$  on a negative example ( $y = 0$ ).
- (iv) Compute the average BCE cost  $J$  over 3 examples:  $Y = (1, 0, 0)$ ,  $\hat{Y} = (0.1, 0.2, 0.7)$ .

**Q5 (Short answer, concept check).** (a) In backpropagation, why is the chain rule unavoidable rather than just a convenience? (b) A friend writes the hidden-layer delta as  $\delta_{h_j} = \delta_{\text{out}} \cdot h_j(1 - h_j)$ , dropping the weight  $w_{jk}$  term. Explain what is wrong with this.

**Q6 (Concept check, ReLU).** A hidden layer uses ReLU. Neuron  $A$  has pre-activation  $a_A = -0.4$ ; neuron  $B$  has pre-activation  $a_B = 0.7$ . Both receive the same downstream contribution  $\sum_k \delta_k w_{jk} = 0.05$ . Find  $\delta_A$  and  $\delta_B$ . What does the difference illustrate, and why is this specific failure mode not a concern for sigmoid/tanh hidden units?

### 3 Answers

**A1.** Forward pass:

$$a_{h_1} = 0.2(0.6) + 0.4(0.1) + 0.3 = 0.46, \quad h_1 = \sigma(0.46) = 0.6130$$

$$a_{h_2} = 0.1(0.6) + 0.5(0.1) + 0.3 = 0.41, \quad h_2 = \sigma(0.41) = 0.6011$$

$$a_y = 0.3(0.6130) + 0.6(0.6011) + 0.2 = 0.7446, \quad \hat{y} = \sigma(0.7446) = 0.6780$$

Loss (not required, shown for context):  $L = \frac{1}{2}(0.3 - 0.6780)^2 = 0.0714$ .

*Backward pass.* Output delta:

$$\delta_{\text{out}} = (\hat{y} - y) \hat{y}(1 - \hat{y}) = (0.3780)(0.6780)(0.3220) = 0.0825$$

Output-layer gradients:

$$\frac{\partial L}{\partial W_7} = \delta_{\text{out}} h_1 = 0.0825(0.6130) = 0.0506, \quad \frac{\partial L}{\partial W_8} = \delta_{\text{out}} h_2 = 0.0825(0.6011) = 0.0496$$

$$W_7^{\text{new}} = 0.3 - 0.1(0.0506) = \boxed{0.2949}, \quad W_8^{\text{new}} = 0.6 - 0.1(0.0496) = \boxed{0.5950}$$

Hidden delta for  $h_2$  (only path needed for  $W_4$ ):

$$\delta_{h_2} = \delta_{\text{out}} W_8 h_2(1 - h_2) = 0.0825(0.6)(0.6011)(0.3989) = 0.01187$$

$$\frac{\partial L}{\partial W_4} = \delta_{h_2} x_1 = 0.01187(0.6) = 0.00713$$

$$W_4^{\text{new}} = 0.1 - 0.1(0.00713) = \boxed{0.09929}$$

**A2.** Forward:  $a_h = w_1 x_1 + w_2 x_2$ ,  $h = \sigma(a_h)$ ;  $a_y = w_3 h$ ,  $\hat{y} = \sigma(a_y)$ ;  $L = \frac{1}{2}(y - \hat{y})^2$ .

*Generic term.* For any weight  $w_i$  feeding pre-activation  $a$  which produces post-activation used downstream, the chain rule gives

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_y} \cdot \frac{\partial a_y}{\partial (\cdot)} \dots \frac{\partial (\cdot)}{\partial w_i}$$

i.e. multiply the local derivative at every node on the path from  $w_i$  to the loss.

Let  $\delta_y = (\hat{y} - y) \hat{y}(1 - \hat{y})$  (output delta). Then:

$$\text{grad}(w_3) = \frac{\partial L}{\partial w_3} = \delta_y \cdot h$$

For  $w_1, w_2$ , continue the chain back through  $w_3$  and the hidden activation:

$$\delta_h = \delta_y w_3 h(1 - h)$$

$$\text{grad}(w_1) = \delta_h \cdot x_1 = \delta_y w_3 h(1 - h) x_1, \quad \text{grad}(w_2) = \delta_h \cdot x_2 = \delta_y w_3 h(1 - h) x_2$$

**A3.** (a) Input→hidden layer: each of  $k$  hidden neurons has  $n$  weights + 1 bias  $\Rightarrow k(n + 1)$ .  
Hidden→output layer: each of  $m$  output neurons has  $k$  weights + 1 bias  $\Rightarrow m(k + 1)$ .

$$\# \text{params} = k(n + 1) + m(k + 1)$$

(b)  $n = 4, k = 5, m = 2$ :  $5(5) + 2(6) = 25 + 12 = \boxed{37}$ .

**A4.** (i) With a sigmoid output, MSE's gradient carries a factor  $\hat{y}(1 - \hat{y})$  that  $\rightarrow 0$  when the prediction is confidently wrong (saturated), so a badly-wrong prediction barely gets corrected; MSE also makes the loss surface non-convex for classification, encouraging poor local minima.

(ii)  $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ .

(iii)  $y = 0, \hat{y} = 0.9$ :  $L = -\log(1 - 0.9) = -\log(0.1) = \boxed{2.303}$  (natural log).

(iv) Per-example losses:  $y = 1, \hat{y} = 0.1 \Rightarrow -\log(0.1) = 2.303$ ;  $y = 0, \hat{y} = 0.2 \Rightarrow -\log(0.8) = 0.223$ ;  $y = 0, \hat{y} = 0.7 \Rightarrow -\log(0.3) = 1.204$ .

$$J = \frac{2.303 + 0.223 + 1.204}{3} = \frac{3.730}{3} = \boxed{1.243}$$

**A5.** (a) Loss is a composition of nested functions (layer after layer of linear map + nonlinearity); there is no way to get  $\partial L / \partial w$  for an early-layer weight without differentiating through every function it passes through, which *is* the chain rule — it isn't optional bookkeeping, it's the only way multivariate composed derivatives work.

(b) Dropping  $w_{jk}$  throws away exactly the information that says *how much* the hidden neuron's output actually influenced the final prediction. A hidden neuron connected by a tiny weight should get a tiny share of the blame; without  $w_{jk}$  every hidden neuron downstream of the same output would incorrectly get the same delta regardless of how strongly it's connected.

**A6.**  $\varphi'(a) = 1$  if  $a > 0$ , else 0.

$$\delta_A = 0.05 \times \varphi'(-0.4) = 0.05 \times 0 = \boxed{0}, \quad \delta_B = 0.05 \times \varphi'(0.7) = 0.05 \times 1 = \boxed{0.05}$$

Neuron  $A$  received a perfectly good downstream signal but propagates *no* gradient at all to any weight feeding into it — it is a "dead" ReLU for this example; it will not update no matter how useful it could have been. Sigmoid/tanh never hit exactly 0 derivative for a finite input (only asymptotically as  $|a| \rightarrow \infty$ ), so a unit can be squashed and learn slowly, but it is never permanently and exactly dead the way a ReLU unit stuck at  $a < 0$  is.